Web Dev with React: useEffect & APIs

Understanding Live vs Static Data, API Waiters, and Automatic Effects

Module: Frontend React Basics

Topic: Fetching Live Data & Effect Hook

Student Name: _____ **Date:** _____

1. CONCEPT 1. Static vs. Live Data: What's the Difference?

Have you ever noticed how some apps change constantly while others stay fixed?

📸 Static Apps

Show fixed, old data that doesn't change unless code is updated manually. It's like looking at a printed photo! A static weather app shows 25°C all day long.

📹 Live Apps

Fetch fresh data dynamically from the internet in real-time. It's like a live video call! Apps like Swiggy, Zomato, or live weather trackers update status automatically.

2. METAPHOR 2. What is an API? Meet the Waiter!

API stands for *Application Programming Interface* — your official data delivery hero!

🍽️ The Restaurant Waiter Analogy

When you sit at a restaurant, you don't walk into the kitchen to cook or grab raw ingredients. You order from the waiter. The waiter takes your request to the kitchen (server) and brings back your delicious food (data)!

- **You** / **App:** The customer making a request.
- **API:** The waiter carrying requests back and forth.
- **Server** / **Database:** The kitchen where all raw data is stored.

3. HOOK 3. The useEffect Hook — The School Bell Alarm!

In React, component functions re-run whenever state updates. What if we want code to run automatically when the page loads, without waiting for button clicks?

🔔 The School Bell Analogy

`useEffect` works just like a school bell ringing automatically at the start of class! Nobody has to press a button — it triggers on its own as soon as school starts (when the component mounts).

```
// The Empty Dependency Array []
useEffect(() => {
  console.log("Page loaded automatically!");
}, []); // [] means: Run ONLY ONCE when page loads!
```

4. CODE PATTERNS Comprehensive React useEffect & API Code Examples

Here are key patterns for fetching live data, handling dependencies, search inputs, error states, and async/await in React.

Example 1: Basic Data Fetching with Loading & Error States

```
import React, { useState, useEffect } from 'react';

function MotivationCard() {
  const [fact, setFact] = useState("");
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch('https://api.planrs.in/grade5/fun-facts')
      .then(res => {
        if (!res.ok) throw new Error('Failed to fetch data');
        return res.json();
      })
      .then(data => {
        setFact(data.fact);
        setLoading(false);
      })
      .catch(err => {
        setError(err.message);
        setLoading(false);
      });
  }, []); // [] = Run once on initial render

  if (loading) return <p>⏳ Calling API Waiter...</p>;
  if (error) return <p>❌ Error: {error}</p>;

  return <div className="card">💡 {fact}</div>;
}
```

Example 2: Async / Await Pattern (Clean Modern Style)

```
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    // Define async function inside effect
    async function loadUserData() {
      try {
        const response = await fetch(`https://api.example.com/users/${userId}`);
        const data = await response.json();
        setUser(data);
      } catch (err) {
        console.error("Waiter failed to get user:", err);
      }
    }
    loadUserData();
  }, [userId]); // Re-runs whenever 'userId' prop changes!

  return user ? <h3>Welcome, {user.name}!</h3> : <p>Loading user...</p>;
}
```

Example 3: Dynamic Search / Live Weather Tracker (Re-fetching on State Change)

```
function WeatherBadge() {
  const [city, setCity] = useState("Mumbai");
  const [weather, setWeather] = useState(null);

  useEffect(() => {
    fetch(`https://api.weather.com/v1/city/${city}`)
      .then(res => res.json())
      .then(data => setWeather(data.temp));
  }, [city]); // Re-runs automatically whenever city changes!

  return (
    <div>
      <button onClick={() => setCity("Delhi")}>Switch to Delhi</button>
      <button onClick={() => setCity("Bengaluru")}>Switch to Bengaluru</button>
      <h3>Weather in {city}: {weather ? `\${weather}°C` : "Loading..."}</h3>
    </div>
  );
}
```

Example 4: Timer & Cleanup Function (Preventing Memory Leaks)

```
function LiveClock() {
  const [time, setTime] = useState(new Date().toLocaleTimeString());

  useEffect(() => {
    const timerId = setInterval(() => {
      setTime(new Date().toLocaleTimeString());
    }, 1000);

    // Cleanup function runs when component unmounts
    return () => clearInterval(timerId);
  }, []);

  return <div className="clock">⏰ Live Time: {time}</div>;
}
```

> ⚠️ **Common Pitfall: Missing Dependency Array**
>
> If you omit the array in `useEffect(() => { ... })`, your effect will trigger on **EVERY single render**. If you update state inside, it creates an **infinite loop** that floods the API server with millions of requests!

5. PRACTICE Student Worksheet & Assessment

Question 1 (Multiple Choice)

What is the main difference between static data and live data in web applications?

[ ] A. Static data updates automatically every second, while live data never changes.

[ ] B. Static data remains fixed like a printed photo, while live data is dynamically fetched fresh like a video call.

[ ] C. Static data comes from an API waiter, while live data is hardcoded into HTML.

[ ] D. There is no difference between static data and live data.

Question 2 (Multiple Choice)

In the restaurant analogy discussed in class, what role does an API play?

[ ] A. The API is the customer ordering food.

[ ] B. The API is the chef in the kitchen preparing data.

[ ] C. The API is the waiter taking requests to the server and bringing data back to the app.

[ ] D. The API is the dining table holding the components.

Question 3 (Multiple Choice)

What happens if you provide an empty dependency array `[]` to a `useEffect()` hook?

[ ] A. The effect code runs continuously in an infinite loop.

[ ] B. The effect code never runs at all.

[ ] C. The effect code runs exactly once when the component first mounts.

[ ] D. The effect code runs only when a button is clicked.

Question 4 (Multiple Choice)

What happens if you accidentally forget to include the dependency array in `useEffect(() => { ... })`?

[ ] A. The code runs only once when the page loads.

[ ] B. The code runs on every single render, potentially causing infinite API calls.

[ ] C. React throws a syntax error and stops working.

[ ] D. The API waiter disappears from the web page.

Question 5 (Short Answer)

Explain in your own words why `useEffect()` is compared to a **School Bell Alarm**.

Question 6 (Code Identification)

Look at the code snippet below. Identify where the API Waiter is called, and explain what `deps` array `[userId]` does:

```
useEffect(() => {
  fetch(`https://api.example.com/user/\${userId}`)
    .then(res => res.json())
    .then(data => setUser(data));
}, [userId]);
```

Question 7 (Homework Code Challenge)

Write or describe a React component using `useEffect` and `fetch` that displays a **Live Weather Badge** for a selected city. Outline the steps from page load to rendering temperature on screen.